LUMINOS Adaptive Window System

Coding Design Decisions

AERO 636 — Spacecraft Human Factors
Texas A&M University

Document covers: Arduino firmware, Python Tapo bridge server, and React dashboard front-end

Authors: Jaden Behringer, Gavin

Date: April 2026

Abstract

This document records the key software and firmware engineering decisions made during the development of the LUMINOS prototype — a subscale adaptive lighting system designed to improve circadian entrainment for ISS crew members. The system spans three software layers: (1) an Arduino microcontroller that reads an analog light-dependent resistor and drives a servo-actuated blue-light gel filter; (2) a Python HTTP bridge server that translates sensor data into closed-loop brightness commands sent to a TP-Link Tapo L530 smart bulb; and (3) a React web dashboard that provides real-time visualization, manual override, and a library of circadian lighting profiles. For each layer, this document explains what was chosen, what alternatives were considered, and why the chosen approach best satisfied the project's constraints of low cost, rapid prototyping, and demonstrable lux-control performance.

Contents

1 System Architecture Overview

The LUMINOS prototype is organized into three loosely coupled software layers, each running on a distinct compute platform and communicating over well-defined interfaces:

1. **Arduino firmware** (`combined_lightservo_test.ino`) — runs on an Arduino Uno or Nano. Reads a GL5528-style LDR via the onboard 10-bit ADC, computes lux using a power-law model, and accepts serial commands to position a servo-actuated blue-light gel filter. Outputs structured sensor data over USB-serial at 9600 baud.
2. **Python bridge server** (`Tapo_cycle_dashboard.py`) — runs on the host laptop. Receives lux readings from the React front-end, implements a closed-loop brightness controller, and issues set-point commands to the TP-Link Tapo L530 smart bulb over Wi-Fi using the `tapo` Python library. Exposes a local REST HTTP API on `127.0.0.1:8765`.
3. **React dashboard** (`arduino-ui/`) — runs in the browser. Reads the Arduino's serial stream via the Web Serial API, forwards lux values to the bridge server, and renders live charts, lighting profile presets, servo/filter controls, and status indicators.

This layered design was chosen so that each subsystem could be developed and tested independently. The Arduino can be validated with a serial monitor before any web code exists; the bridge server can be exercised with `curl` before the dashboard is built; and the dashboard can run in a "simulation mode" (no Arduino connected) to allow UI development in parallel.

Table 1: Inter-layer communication interfaces

From	To	Interface
Arduino	React dashboard	USB-Serial, 9600 baud, text lines
React dashboard	Python bridge	HTTP REST (localhost:8765)
Python bridge	Tapo L530 bulb	Wi-Fi LAN, Tapo cloud API (SDK)
React dashboard	Arduino	USB-Serial write (servo angle commands)

2 Arduino Firmware

2.1 Light Sensor Hardware and Circuit

2.1.1 Sensor selection: GL5528 LDR

A GL5528-style light-dependent resistor (LDR) was chosen over more expensive alternatives (e.g., BH1750 digital ambient-light sensor, TSL2561) because it costs under $1, requires no I^2C library, and the goal was a proof-of-concept lux demonstration rather than research-grade photometry. The trade-off is that the GL5528's response is nonlinear and device-to-device variation is high; however, because we perform a one-point calibration (see Section 2.2), absolute accuracy is adequate for our purpose.

2.1.2 Voltage-divider circuit

The LDR is wired in a resistor divider with a fixed 10 kΩ series resistor to ground. The junction voltage is read on analog pin `A0`:

$$V_{\text{out}} = V_{CC} \cdot \frac{R_{\text{fixed}}}{R_{\text{LDR}} + R_{\text{fixed}}}$$

With $V_{CC} = 5$ V and $R_{\text{fixed}} = 10\,000\ \Omega$, the mid-range sensitivity falls around typical indoor illuminance levels (100–500 lux), which matches the project's operating range.

Divider orientation choice: The LDR is placed on the high side (VCC) and the fixed resistor on the low side (GND), so that *more light → lower LDR resistance → higher ADC reading*. This is the conventional orientation for this component and was confirmed during the `light_level_test` calibration phase. (An earlier wiring attempt reversed this and required a corrected formula in firmware — the sign of the inversion is documented in `light_level_test.ino`.)

2.2 Lux Conversion Model

2.2.1 Power-law model

The GL5528 datasheet specifies that resistance follows an approximate power law in lux:

$$R_{\text{LDR}} = R_{10} \left(\frac{E}{10} \right)^{-\gamma} \quad \Longrightarrow \quad E = 10 \cdot \left(\frac{R_{10}}{R_{\text{LDR}}} \right)^{1/\gamma}$$

where E is illuminance in lux, R_{10} is the LDR resistance at 10 lux, and γ is the slope exponent. In firmware this becomes:

```
const float GAMMA = 0.7;
const float R10   = 20000.0;   // LDR resistance at 10 lux (ohms)

float rawToLuxPowerLaw(int raw) {
  if (raw <= 0)   return 0.0;
  if (raw >= 1023) raw = 1022; // avoid divide-by-zero

  float vOut = raw * (VCC / 1023.0);
  float rLdr = R_FIXED * ((VCC / vOut) - 1.0);
  if (rLdr <= 0.0) return 0.0;

  return 10.0 * pow(R10 / rLdr, 1.0 / GAMMA);
}
```

Listing 1: Power-law lux conversion in firmware

2.2.2 Parameter choices: γ and R_{10}

- $\gamma = 0.7$ is taken from the GL5528 typical datasheet value. No multi-point calibration was performed; the default was judged adequate for the relative-lux comparisons the prototype makes.
- $R_{10} = 20\,000\ \Omega$ was empirically tuned during the `light_level_test` session to match the observed ADC output under an approximately known illuminance (room overhead light). The firmware prints a "cal hint" at startup to help future calibration: `ADC at 10 lux should be near <value>`.

2.2.3 Edge-case protection

Two guard conditions are included: `raw` $\leq$ `0` returns 0.0 (sensor shorted or completely dark), and `raw` $\geq$ `1023` is clamped to 1022 to prevent division by zero in the V_{out} calculation. These are defensive choices that avoid undefined floating-point behavior on the AVR.

2.3 Sensor Sampling Loop

2.3.1 Non-blocking 100 ms interval

The sensor is sampled every 100 ms using a `millis()`-based timestamp comparison rather than `delay()`. This keeps the main loop responsive to incoming serial commands even during sensor reads:

```
unsigned long lastSensorRead = 0;
const int SENSOR_INTERVAL = 100; // ms

void loop() {
  if (millis() - lastSensorRead >= SENSOR_INTERVAL) {
    lastSensorRead = millis();
    readLightSensor();
  }
  // ... serial command handling follows
}
```

Listing 2: Non-blocking sensor sampling

Why 100 ms: The bridge server's control loop runs at ∼40 ms, and the React dashboard polls at 70 ms. A 100 ms sample rate provides marginally faster data than the consumer; faster rates were tried but produced no perceptible improvement in control quality and increased serial bus load.

2.3.2 Serial output format

Each sensor reading is emitted as a single structured text line:

`RAW:647,VOLTAGE:3.17,LUX:284.5`

This comma-key format was chosen so that the React frontend can parse it with a single regular expression (`/RAW:\s*(\d+),VOLTAGE:...,LUX:.../i`) without a full parser library. The field names are explicit to facilitate manual inspection via a serial monitor and to make parsing order-independent.

2.4 Servo Control

2.4.1 Microsecond-based positioning

All servo commands are issued via `writeMicroseconds()` rather than the simpler `write(angle)` API. This was chosen for two reasons:

1. The `write()` function maps 0–180° to a library-internal pulse range that varies between servo models. Specifying microseconds directly removes this ambiguity and was found necessary to reach the physical end-stops of the particular SG90 unit used.
2. It enables fine-grained nudge commands (± 10 μs or ± 50 μs) that would be impossible to express in integer degrees.

2.4.2 Hard safety clamp

The firmware enforces a pulse-width clamp of [725, 2310] μs on every move, regardless of command source:

```
void moveToMicroseconds(int us) {
  us = constrain(us, 725, 2310);
  // ...
}
```

Listing 3: Safety clamp on servo pulse width

The range was determined empirically: values outside this range caused audible stalling on the SG90 test unit. The clamp lives inside `moveToMicroseconds` so that every code path (angle commands, nudge commands, serial input) is protected by the same guard.

2.4.3 Angle-to-microsecond mapping

When a user sends an integer angle (0–180), it is mapped to the microsecond range with `map(angle, 0, 180, 740, 2290)`. The slightly inset range (740–2290 vs. 725–2310) provides a small margin of comfort before hitting the hard clamp. This was chosen after observing that the servo buzzed at the absolute limits.

2.4.4 Lazy attach / detach pattern

The servo is only attached to its PWM pin when a move command is received, and a `detach` command releases it:

```
void moveToMicroseconds(int us) {
  if (!isAttached) {
```

```
    myServo.attach(SERVO_PIN);
    isAttached = true;
  }
  myServo.writeMicroseconds(us);
}
```

Listing 4: Lazy attach pattern

Motivation: When the Arduino library holds a servo attached and idle, it continues driving the PWM signal which causes an audible 50 Hz buzz and unnecessary current draw. Detaching when not in use eliminates this. The dashboard's "detach" serial command makes this user-accessible.

2.4.5 Window filter angle mapping

The React dashboard's window filter slider (0–100%) is translated to a servo angle by the formula:

$$\theta = 180 - \left(\frac{f}{100}\right) \times 180$$

where f is the filter percentage. At $f = 0\%$ (no filter) the servo is at 180°; at $f = 100\%$ (fully filtered) it is at 0°. This convention was chosen so that increasing the "filter" value physically swings the gel panel into the light path.

3 Python Bridge Server

3.1 Architecture: Threading + Asyncio

3.1.1 Why a separate bridge process?

The Tapo L530 is controlled via an async Python SDK (`tapo`). Embedding this directly in the React application is impossible (browser sandbox), and running it in a standard synchronous HTTP handler would block the server during every bulb command (∼200–800 ms per command). The bridge server solves this by decoupling the fast HTTP layer from the slow async bulb API.

3.1.2 Dedicated asyncio thread

The controller spawns a single daemon thread running its own asyncio event loop:

```
self.loop = asyncio.new_event_loop()
self._thread = threading.Thread(target=self._loop_thread, daemon=True)
self._thread.start()

def _loop_thread(self):
    asyncio.set_event_loop(self.loop)
```

```
    self.loop.create_task(self._control_loop())
    self.loop.run_forever()
```

Listing 5: Asyncio loop in dedicated thread

Synchronous callers (HTTP handlers) submit coroutines to this loop via `asyncio.run_coroutine_threadsa` and block with an 8-second timeout. This keeps the HTTP server non-blocking and ensures all Tapo I/O stays on a single event loop. An 8-second timeout was chosen because Tapo bulb commands typically complete in under 500 ms; 8 seconds provides a generous margin for transient Wi-Fi congestion.

3.1.3 ThreadingHTTPServer

The HTTP layer uses `ThreadingHTTPServer` (from `http.server`) so that concurrent requests from the React dashboard (state polling plus sensor push, each on a 70–120 ms interval) do not serialize. This was simpler than introducing an async HTTP framework like `aiohttp` and adequate for a local-only server with at most 2–3 simultaneous connections.

3.2 Connection Management

3.2.1 Retry logic with configurable timeout

Connecting to the Tapo bulb over Wi-Fi is unreliable; the SDK occasionally times out even when the bulb is on the same subnet. Three retry attempts with a 10-second per-attempt timeout are the defaults (all overridable via environment variables). Between retries a 1-second sleep is inserted to give the bulb time to recover from a failed handshake.

3.2.2 Pre-connection diagnostics

Before each connection attempt the controller runs a lightweight network diagnostic:

```
def _build_connect_diagnostics(self, ip):
    # DNS resolution, local route IP, TCP port 80/443 reachability
    ...
```

Listing 6: Network diagnostics before connect

This logs: hostname, resolved IP, local source IP for the route, and whether TCP ports 80 and 443 are open on the target. These diagnostics proved essential during development when the bulb had a DHCP address change and the log entry immediately identified that port 80 was unreachable.

3.2.3 Lazy device handle

The `_device` reference is set to `None` on any connection failure and recreated on the next command via `_ensure_device()`. This "lazy reconnect" pattern avoids the need for a separate reconnection coroutine and ensures that a transient drop automatically heals on the next scheduled brightness command.

3.2.4 Connection verification before enabling auto mode

When the user enables auto-control from the dashboard, the bridge server performs a blocking `_ensure_device()` call before setting `autoEnabled = True`. If the bulb cannot be reached, auto mode is rejected and an error is returned to the dashboard. This prevents the system entering a silent failure state where it believes it is controlling the bulb but is not.

3.3 Closed-Loop Brightness Controller

3.3.1 Control loop execution model

The controller runs as a persistent asyncio coroutine sleeping for `control_interval_s = 0.04 s` (25 Hz) between iterations. Each tick it checks whether new sensor data has arrived (via `_last_sensor_ts` vs. `_last_controlled_sensor_ts`) before acting, ensuring the controller never issues a command based on stale data.

3.3.2 Moving average sensor filter

Raw lux readings from the Arduino (forwarded via the React bridge) are stored in a `collections.deque` of length 6 and averaged before computing the control error:

```
self._lux_samples = deque(maxlen=self.moving_avg_window)  # window = 6
# ...
avg_lux = sum(lux_samples) / len(lux_samples)
error   = target - avg_lux
```

Listing 7: Moving average smoothing

A window of 6 was chosen empirically: smaller windows (2–3) caused excessive step commands due to ADC noise; larger windows (10+) introduced perceivable lag when lux changed quickly. The controller also waits until the deque is full before issuing any command, to avoid acting on an unrepresentative initial sample.

3.3.3 Exponential step curve

Brightness is adjusted in integer steps rather than by a continuous PID output (because the Tapo API only accepts integer 1–100 brightness values). The step size follows an exponential curve keyed on the magnitude of the lux error:

$$\text{step} = \begin{cases} \delta_{\min} & \text{if } |\epsilon| \leq \epsilon_{\text{small}} \\ \delta_{\min} + (\delta_{\max} - \delta_{\min}) \cdot \left(1 - e^{-k(\,|\epsilon| - \epsilon_{\text{small}}\,)}\right) & \text{otherwise} \end{cases}$$

with parameters: $\delta_{\min} = 1$, $\delta_{\max} = 5$, $\epsilon_{\text{small}} = 100$ lux, and gain $k = 0.02$.

Design rationale: A fixed step caused either oscillation near the target (step too large) or sluggish response far from it (step too small). A linear ramp was tried first but still oscillated;

the exponential curve reaches a plateau quickly and avoided oscillation while maintaining fast convergence from large initial errors.

Why $\delta_{\max} = 5$: Larger values (tried: 10, 20) caused visible flicker because the bulb takes ∼200 ms to physically respond to a brightness change, and a 20-unit jump followed by an overshoot step 800 ms later produced noticeable flicker. Capping at 5 units per step smoothed this completely.

3.3.4 Reverse damping

When the controller direction reverses (e.g., switches from incrementing to decrementing brightness), a 3-second damping window is entered during which the step is capped at 1:

```
if self._last_direction != 0 and direction != self._last_direction:
    self._reverse_damp_until_ts = now + self.reverse_damping_s  # 3.0 s
    step = self.min_step
elif now < self._reverse_damp_until_ts:
    step = min(step, max(self.min_step, 2))
```

Listing 8: Reverse damping logic

Motivation: Direction reversals are a primary source of oscillation in bang-bang and near-dead-band controllers. When the controller overshoots and reverses, the damping window prevents it from immediately over-correcting in the opposite direction.

3.3.5 Two-tier command intervals

Two different inter-command delays are used:

- **Normal:** 0.8 s — the Tapo bulb's measured response latency from command receipt to stable light output.
- **Close-to-target** ($|\epsilon| \leq 70$ lux): 2.5 s — extra settling time when near the set-point, where small lux fluctuations could otherwise trigger rapid small-step commands that add up to overshoot.

Both values were tuned by observation: the 0.8 s base interval matched the measured bulb response time; the 2.5 s close-interval eliminated the last remaining oscillatory pattern seen near the target in bench testing.

3.3.6 Fixed color temperature

During auto-control the bulb's color temperature is fixed at 4000 K. This value was chosen as a neutral "office white" that is appropriate for daytime activity phases and does not interfere with the brightness-based lux control. Circadian color temperature scheduling (warm evening / cool daytime) is handled at the profile layer in the React dashboard and falls outside the scope of the closed-loop controller.

3.4 HTTP REST API

3.4.1 Endpoint design

Table 2: Bridge server REST endpoints

Method	Path	Purpose
GET	`/state`	Return full controller state JSON
GET	`/health`	Alias of `/state` for health checks
POST	`/sensor`	Push a lux reading from the Arduino
POST	`/target`	Update the target lux set-point
POST	`/auto`	Enable or disable auto-control
POST	`/brightness`	Manual brightness override (1–100)
POST	`/power`	Power the bulb on or off

The endpoints were designed to be minimal and stateless: every response includes the complete current state so the client does not need to make a separate `/state` call after a mutation.

3.4.2 CORS headers

All responses include `Access-Control-Allow-Origin: *` and a preflight `OPTIONS` handler. This was necessary because the React dashboard is served from a `localhost` origin different from the server's port (8765) and modern browsers enforce cross-origin restrictions even on localhost.

3.5 Logging

A structured logger (`tapo_bridge` namespace) writes to both the console and a rotating log file (`tapo_bridge.log`). Log level and file path are configurable via environment variables. Key events logged: connection attempts with full diagnostics, every brightness command issued, auto-mode state transitions, and all exceptions with tracebacks. This level of logging was essential during bench tuning to distinguish controller instability from Wi-Fi instability.

3.6 Simple Cycle Script

The companion script `Tapo_cycle.py` is an early standalone test that cycles brightness and color temperature through 18 linear steps using `numpy.linspace`:

```
warm  = np.concatenate([np.linspace(2500, 5000, 9),
                        np.linspace(5000, 2500, 9)])
light = np.concatenate([np.linspace(10, 100, 9),
                        np.linspace(100, 10, 9)])
```

Listing 9: NumPy linspace cycle

This script has no HTTP server or control loop; it was used purely to verify Tapo connectivity and the "feels" of the brightness/warmth transition before the full closed-loop system was built.

4 React Dashboard Front-End

4.1 Architecture and State Management

The dashboard uses two custom React hooks as its data layer:

- `useLiveData` — manages the Web Serial API connection to the Arduino, parses incoming lux data, maintains simulation state when no Arduino is connected, and computes derived display quantities (orbit phase, mission time, alert level, active profile).
- `useTapoBridge` — manages all communication with the Python bridge server, exposing bridge state and action functions to UI components.

These were separated from component logic to keep UI components pure and to enable independent testing. The `Dashboard` component is the only component that imports both hooks; all child components receive only the data they need via props.

4.2 `useLiveData`: Web Serial API

4.2.1 Web Serial over a dedicated library

The Web Serial API was used directly rather than through a wrapper library. This was chosen because the API surface needed is small (open, read lines, write strings) and introducing a dependency would have complicated the build.

4.2.2 Auto-reconnect on mount

On component mount the hook calls `navigator.serial.getPorts()` to check for previously granted port permissions. If a port is found, it connects immediately without user interaction. This avoids requiring the user to click "Connect Arduino" on every page reload during development.

4.2.3 TextDecoderStream with fallback

The serial stream is decoded using `TextDecoderStream` when available (Chrome 89+), falling back to a manual `TextDecoder` on older Chromium builds. The fallback handles the case where the embedded test browser does not support the piped stream API.

4.2.4 Line buffering

Incoming serial data is accumulated in a string buffer and split on `\r?\n`. Only complete lines are parsed; partial lines are held in the buffer until the next chunk arrives. This prevents malformed lux readings from corrupted partial lines.

4.2.5 Regex parsing of Arduino output

Each complete line is matched against:

`/RAW:\s*(\d+)\s*,\s*VOLTAGE:\s*([\d.]+)\s*,\s*LUX:\s*([\d.]+)/i`

The case-insensitive flag and optional whitespace in the pattern tolerate minor formatting variation without rewriting the Arduino firmware.

4.2.6 Simulation mode

When no Arduino is connected (`serialConnected === false`), all sensor values are synthesized from a time-based sinusoidal model:

```
const simulatedRaw = Math.round(Math.min(1023, Math.max(0,
  600 + Math.sin(t / 15) * 150
      + Math.sin(t / 5)  * 30
      + Math.random()    * 15
)));
```

Listing 10: Simulated lux in simulation mode

Simulation mode serves two purposes: it allows UI development without physical hardware, and it provides a live-looking display during demonstration when the Arduino is unavailable. The "SIM MODE" status indicator in the top bar clearly distinguishes simulated from live data.

4.2.7 Alert level computation

Three alert levels are computed inline with the sensor update loop:

```
// Three-level alert per NASA SSP_50005 ISS Human Integration Standard
// Emergency (class 1) = solar flare; Caution (class 3) = lux out of tolerance
const alertLevel = solarAlert
  ? 'emergency'
  : (lux > 450 || lux < 50) ? 'caution' : null;
```

Listing 11: Three-level alert per NASA SSP_50005

The threshold values (450 lux upper / 50 lux lower) were chosen to bracket the nominal operating range of 50–400 lux. "Solar alert" is triggered when the lux delta between successive 150 ms ticks exceeds 80 lux, simulating an unexpected solar-flare illumination spike.

4.3 `useTapoBridge`: Bridge Communication

4.3.1 Polling frequency

The hook polls `/state` every 120 ms and pushes sensor data via `/sensor` every 70 ms. These rates were chosen so that:

- 70 ms sensor push ≈ 14 Hz, faster than the bridge's 25 Hz control loop to ensure every control iteration has fresh data.
- 120 ms state poll ≈ 8 Hz, fast enough for the UI to feel responsive to brightness changes without generating unnecessary HTTP traffic.

4.3.2 Temporary auto-control feature

When the user clicks a lighting profile preset, `startTemporaryAuto` is called. This sets the target lux and enables auto-control temporarily:

```
const startTemporaryAuto = useCallback(async (lux) => {
  await setTargetLevel(lux);
  await setAutoEnabled(true);
  setTempAutoTarget(Number(lux));
}, [setTargetLevel, setAutoEnabled]);
```

Listing 12: Temporary auto-control

A `useEffect` watches `bridge.currentLux` and automatically disables auto-control once the lux reaches within 1 lux of tolerance of the target. This allows one-tap profile switching without leaving auto-control permanently enabled, which the user may not want.

4.3.3 Configurable bridge URL

The bridge base URL is read from the `REACT_APP_TAPO_BRIDGE_URL` environment variable, defaulting to `http://127.0.0.1:8765`. This allows the bridge to run on a different machine during lab demos without modifying source code.

4.4 Lighting Profiles

Ten lighting profiles are defined in `src/data/dummy.js`, covering the full range of ISS crew activities:

Table 3: Lighting profiles and target illuminances

Profile	Lux Range	Target Lux	Blue Light Level
SLEEP	0 lux	0	210
DAWN	40–100 lux	70	840
DUSK	80–140 lux	110	620
LEISURE	250–350 lux	300	1050
HYGIENE	400–500 lux	450	1280
MEAL	550–650 lux	600	1420
EXERCISE	750–850 lux	800	2450
MAINTENANCE	950–1050 lux	1000	2820
EVA PREP	1200–1300 lux	1250	3150
RESEARCH	1500+ lux	1550	3400

The target lux values were selected to produce statistically distinguishable illuminance levels that map to common ISS crew activity categories.

4.4.1 Active profile detection

The dashboard highlights whichever profile is nearest to the current measured lux:

```
let activeProfile = PROFILES[0].id;
let minDist = Infinity;
for (const p of PROFILES) {
  const dist = Math.abs(lux - p.targetLux);
  if (dist < minDist) { minDist = dist; activeProfile = p.id; }
}
```

Listing 13: Nearest-profile detection

This provides continuous feedback on which operating mode the current lighting most closely corresponds to.

4.4.2 Blue light level computation

The "blue light level" metric displayed in the control panel is computed from the window filter slider position:

$$B = 3500 - 3300 \cdot \frac{f}{100}$$

where $f \in [0, 100]$ is the filter percentage. At $f = 0\%$ (no filter), $B = 3500$ K equivalent; at $f = 100\%$ (full filter), $B = 200$. This is a display heuristic — not a calibrated measurement — intended to give the user intuitive feedback about the qualitative blue-light intensity of the current lighting environment. The range of 200–3500 was chosen to span from near-zero (amber gel fully in path) to a notional daylight-equivalent.

4.4.3 Window filter simulated lag

The effective filter value shown in the display applies a first-order-like lag with a random 20% hold probability:

```
setEffectiveFilter(prev => {
  const diff = windowFilter - prev;
  if (Math.abs(diff) < 0.5) return windowFilter;
  if (Math.random() < 0.2)  return prev; // hold, simulating mechanical delay
  return prev + diff * (0.15 + Math.random() * 0.2);
});
```

Listing 14: Simulated filter lag

This was added purely as a UX detail to make the filter display feel physically plausible rather than snapping instantly, reflecting that a servo-driven filter panel takes time to traverse.

4.5 Component Structure

Table 4: React component responsibilities

Component	Responsibility
`Dashboard`	Root layout; owns both hooks; routes props
`TopBar`	Status indicators (serial, bridge, bulb); MET clock
`TapoControlPanel`	Main control card; target lux, brightness, auto toggle
`LightChart`	60-second rolling lux history (Recharts `AreaChart`)
`ProfilesPanel`	Lighting profile grid; active profile highlight
`CrewPanel`	Crew member activity cards (static for prototype)
`AlertBar`	Three-level (nominal/caution/emergency) status bar
`LightGauge`	Circular lux gauge (supplementary display)

4.6 LightChart: 60-Second Rolling Window

The lux history is maintained as a fixed-length array of 60 values, updated by slicing off the oldest entry and appending the newest:

```
histRef.current = [...histRef.current.slice(1), lux];
```

Listing 15: Rolling history update

The 60-second window (one sample per 150 ms update tick $\approx$ 400+ samples, but stored as 60 discrete lux values separated by 1-second intervals for display clarity) was chosen to show trend context without overwhelming the chart. Animation is disabled on the `Area` component (`isAnimationActive=false`) to prevent jitter during high-frequency updates.

5 Hardware Component Selection

This section records component choices that directly influenced software design decisions.

Table 5: Hardware components and selection rationale

Component	Selected	Rationale
Smart bulb	TP-Link Tapo L530	Supports independent brightness and color-temperature control; well-maintained Python SDK; avoids custom LED wiring complexity; ∼$15
Blue-light filter	Rosco theater gel (amber/orange, e.g., Roscolux #312)	Purpose-built for absorbing blue wavelengths; cheap ($2–3/sheet); easy to cut and mount on servo arm; produces measurable ΔCCT on phone lux meter
Filter actuator	SG90 micro servo	∼$3/unit; ample torque for a gel panel; standard Arduino library support; 180° range maps cleanly to open/closed filter positions
Light sensor	GL5528 LDR	<$1; no I^2C overhead; power-law model well documented; adequate precision for relative lux comparison
Microcontroller	Arduino Uno/Nano	USB-serial built in; 10-bit ADC; native Servo library; large support community; ∼$5–15

5.1 Active vs. Passive Filtration

Three filtration approaches were considered:

1. **PDLC smart film** — electrically switchable opaque/transparent. Rejected because small samples require 60–120 V AC and a driver circuit, adding cost and complexity outside the project's scope.
2. **Electrochromic film** — voltage-variable tinting. Rejected because small-quantity sourcing is difficult and cost is high; this was already flagged as a risk in the preliminary design report.
3. **Servo-actuated gel panel** (chosen) — mechanical swing of a physical gel filter into the light path. Low cost, simple control (one PWM pin), directly observable, and maps cleanly to the three-layer software architecture.

6 Known Limitations and Future Work

1. **Absolute lux accuracy:** The GL5528 power-law model is a single-point calibration with a fixed γ from the datasheet. A multi-point calibration against a reference lux meter would improve accuracy, which matters if the system is used for quantitative circadian dose calculations.
2. **Bulb latency jitter:** The 0.8 s command interval was tuned for one specific L530 unit over one specific Wi-Fi network. On a congested network the latency can exceed this, leading to over-shoot. Future work could measure round-trip latency per command and adapt the interval dynamically.
3. **Color temperature scheduling:** The current software fixes color temperature at 4000 K during closed-loop control. A full implementation would vary CCT with time-of-day as part of the circadian profile.
4. **Multi-bulb support:** The `IP_ADDRESS_LIST` field already supports multiple IP addresses but multi-bulb simultaneous control has not been tested. The current architecture would serialize commands across bulbs, which would need to become concurrent for a real implementation.
5. **Web Serial API browser support:** The current implementation requires Chromium-based browsers. Firefox and Safari do not support the Web Serial API. A fallback WebSocket bridge from a local serial daemon would be required for broader browser support.

7 Summary of Key Design Decisions

Decision	Choice Made	Primary Reason
Lux model	Power-law with $\gamma = 0.7$, $R_{10} = 20$ kΩ	Matches GL5528 datasheet; single-point calibration sufficient
Servo positioning	`writeMicroseconds()`	Model-independent; enables ± 10 μs fine nudge
Servo attach/detach	Lazy attach, user-accessible detach command	Eliminates idle buzz and current draw
Serial format	`RAW:x,VOLTAGE:x,LUX:x`	Human-readable; single-regex parseable
Bridge threading	asyncio loop in daemon thread	Decouples slow Tapo I/O from fast HTTP handler
Sensor smoothing	Moving average, window = 6	Empirically removes ADC noise without excess lag

(continued)

Decision	**Choice Made**	**Primary Reason**
Brightness step curve	Exponential clipped to [1, 5]	Eliminates flicker from large steps; fast convergence
Reverse damping	3-second hold at `min_step` on direction flip	Primary oscillation suppressor
Command intervals	0.8 s normal, 2.5 s near target	Matched to measured bulb response latency
Color temperature	Fixed 4000 K during auto	Neutral; does not interfere with lux control
Serial read (React)	Web Serial API with TextDecoderStream	No server needed; browser-native; auto-reconnect
Bridge polling	120 ms state, 70 ms sensor push	70 ms $>$ 25 Hz controller rate; 120 ms responsive UI
Temp auto-control	Auto-disables when target reached	Convenient profile switching without permanent auto mode
Alert thresholds	NASA SSP_50005, 3-level	Consistent with ISS Human Integration Standard
Filter actuation	Servo + Rosco gel	\$3–5 total; mechanically observable; no high-voltage drive